PROJECT REPORT OF CO-OP Project at Industry (Module-2)

ON

Synergistic Approach to Automated Plant Disease Diagnosis

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Shivanshu (2210992329)

Suraj Veer (2210990878)

Supervised by:

Dr. Ajay Kumar

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

DECLARATION

We hereby certify that the work which is being presented in the project report entitled "Synergistic Approach to Automated Plant Disease Diagnosis" in partial fulfilment of requirements for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the Department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of our own work carried out under the supervision of Dr.Ajay Kumar. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura **Shivanshu (2210992329)**

Date: 10/05/2026 **Suraj Veer(2210990878**

This is to certify that the above statement made by the candidates is correct to the best of my knowledge and belief.

Dr. Ajay Kumar

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who supported us throughout this project. We are deeply thankful to our mentor and supervisor for their invaluable guidance, constant encouragement, and constructive feedback during the course of this work.

We extend our heartfelt thanks to the Department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology for providing the necessary infrastructure and resources to carry out this project.

We are also grateful to the developers and contributors of the PlantVillage dataset, open-source deep learning frameworks (PyTorch, FastAPI), and the broader research community whose published works formed the foundation of this research.

Finally, we thank our families and friends for their unwavering moral support throughout this journey.

Shivanshu
Suraj Veer

ABSTRACT

Crop diseases continue to be a persistent threat to global food security, reducing crop yields by 20 to 40 percent every year. Traditional methods of disease diagnosis require specialists to examine plants individually — a process that is slow, expensive, and inaccessible to smallholder farmers in remote regions. This project presents a deep learning-based framework for automatically diagnosing multiple types of plant leaf diseases.

Two approaches were evaluated: object detection using YOLOv8 and image classification using EfficientNet-B0 trained with transfer learning on the PlantVillage dataset. Although YOLOv8 was initially tested for disease localization, it underperformed due to the absence of bounding-box annotations in PlantVillage. EfficientNet-B0 was subsequently adopted as the primary classifier and trained for five epochs under computational constraints, achieving a validation accuracy of 93.1% with rapid convergence and solid generalization.

A comparative evaluation against VGG16, ResNet-50, and MobileNetV2 under identical five-epoch conditions confirmed that EfficientNet-B0 delivers the best accuracy-to-efficiency ratio, using only 5.3 million parameters. The trained model was deployed as a FastAPI web application that accepts a leaf image and returns real-time disease predictions, confidence scores, and treatment recommendations — making the system practically useful for farmers in the field.

Keywords: *plant disease diagnosis, EfficientNet-B0, transfer learning, YOLOv8, deep learning, FastAPI, precision agriculture, PlantVillage dataset.*

LIST OF FIGURES AND TABLES

List of Figures

List of Tables

CONTENTS

1. INTRODUCTION

Agriculture is the backbone of many developing economies, yet it remains perpetually under threat from fungal, bacterial, and viral pathogens. According to the Food and Agriculture Organization, plant diseases wipe out 20–40% of global crop production every year. This is not merely a statistic; it represents a devastating blow for millions of farmers worldwide. Early disease detection is critical — catching disease before it spreads allows farmers to act quickly and limit losses.

Traditional methods of spotting diseases depend on specialists examining plants in person, or laboratory tests such as PCR and ELISA. While effective, these approaches are expensive and time-consuming. Most smallholder farmers in remote areas cannot access them. Missing an early diagnosis often leads to incorrect treatment, wasted resources, or even total crop failure.

With the rise of computer vision, deep learning, and ubiquitous smartphone access, there is now an opportunity to deliver instant disease identification to farmers in the field. This report presents a practical framework integrating high-accuracy deep learning models with a user-friendly web application. The key contributions of this work are: (1) a head-to-head comparison of object detection and image classification for leaf disease analysis; (2) benchmarking of four widely-used CNN architectures under identical five-epoch training conditions; (3) a real web application delivering real-time disease predictions with treatment recommendations; and (4) quantitative evidence that EfficientNet-B0 is parameter-efficient and deployment-ready.

This project was motivated by the real struggles faced by farmers in developing nations — scarce extension services, expensive agrochemicals, and no quick way to diagnose disease. A tool accessible via a basic mobile browser, delivering results in seconds without any specialist or laboratory, represents a meaningful step toward reducing crop losses at the farm level.

2. LITERATURE REVIEW

2.1 Classical Machine Learning Approaches

Early automated disease detection systems relied on handcrafted features — colour histograms, texture descriptors, and shape vectors — fed into classifiers such as Support Vector Machines and k-Nearest Neighbour models. In controlled settings, these performed reasonably well. However, real farm photographs present varying illumination, diverse backgrounds, and unusual leaf angles that these methods could not handle. Furthermore, traditional classifiers struggle when symptoms of different diseases appear visually similar at early stages, causing misclassification of subtle cues.

2.2 Convolutional Neural Networks

Deep CNNs transformed the field. Mohanty et al. [1] demonstrated the effectiveness of AlexNet and GoogLeNet on the PlantVillage dataset for plant disease classification. Subsequent architectures such as VGG16 [3] and ResNet-50 [4] raised the accuracy bar but introduced massive parameter counts — 138 million and 25.6 million respectively — making deployment on resource-constrained devices difficult. MobileNetV2 aimed to reduce this burden using depthwise-separable convolutions, though at a slight cost to accuracy. Ramcharan et al. [8] further demonstrated the viability of fine-tuning CNN models for local crop types in East Africa, highlighting the importance of dataset diversity for real-world applicability.

2.3 Object Detection Approaches

Researchers explored object detection frameworks such as YOLO [5] and Single-Shot Detectors to not only classify but also localize disease spots directly in leaf images. Singh et al. [6] created the PlantDoc dataset specifically for this purpose. However, object detection requires bounding-box annotations, which are costly to produce. In the PlantVillage dataset, disease spots often resemble normal leaf patterns, making false positives common and localization setups unnecessarily complex for what is fundamentally a classification problem.

2.4 Efficient Architectures and Transfer Learning

EfficientNet, introduced by Tan and Le [2], employs compound scaling of depth, width, and resolution using a single scaling coefficient, delivering top-tier accuracy with far fewer parameters than older models. EfficientNet-B0, the baseline variant, has approximately 5.3 million parameters. Atila et al. [7] demonstrated that EfficientNet models perform exceptionally well for plant disease classification even with limited data. This project extends that work by embedding EfficientNet-B0 into a complete deployment pipeline and conducting a rigorous comparison across multiple architectures under uniform training conditions.

3. METHODOLOGY

3.1 Dataset

All experiments were conducted on the PlantVillage dataset, widely regarded as the gold standard for computational plant pathology. It comprises 54,305 RGB images of leaves from 14 crop species, organized into 38 classes encompassing both healthy leaves and leaves exhibiting common diseases. Classes include Apple Scab, Tomato Early Blight, Corn Common Rust, and Grape Black Rot, among others. Standard RGB images (as opposed to segmented versions) were used to better reflect real-world field conditions where users capture unprocessed photographs.

3.2 Preprocessing and Data Augmentation

All images were resized to 224 × 224 pixels to match EfficientNet-B0's input requirements. Pixel values were normalized using ImageNet channel-wise means ([0.485, 0.456, 0.406]) and standard deviations ([0.229, 0.224, 0.225]) for consistency with the model's pre-training conditions. Real-time data augmentation was applied during training to improve generalization and reduce overfitting, including random horizontal flips (50% probability), random rotations of up to ±25°, and brightness jitter with a factor of 0.2. The dataset was divided into 70% training, 15% validation, and 15% test splits, stratified by class.

3.3 Phase 1 — YOLOv8 Object Detection

The first experimental phase treated leaf disease detection as an object detection problem using YOLOv8, with the intention of drawing bounding boxes around diseased regions. Performance was poor, with mean Average Precision at IoU=0.5 never exceeding 25%. The primary challenges were: (1) PlantVillage does not include bounding-box annotations, necessitating the use of imprecise pseudo-labels; (2) significant visual overlap between disease lesions and normal leaf textures generated frequent false positives; and (3) the localization setup added unnecessary complexity to what is inherently a whole-image classification task. YOLOv8 was consequently abandoned in favour of image classification.

3.4 Phase 2 — EfficientNet-B0 with Transfer Learning

A pre-trained EfficientNet-B0 model with ImageNet weights was loaded, and its 1,000-class output head was replaced with a single fully connected layer mapping 1,280 input features to 38 disease class logits. Fine-tuning was performed using the Adam optimizer with a learning rate of 1×10^{-4} and a batch size of 32, with categorical cross-entropy loss. Training ran for five epochs, constrained by available compute resources. Despite this limitation, the model demonstrated rapid convergence and strong domain adaptation, as reported in the results section.

4. TOOLS AND TECHNOLOGIES

4.1 Deep Learning Framework

PyTorch was used as the primary deep learning framework, selected for its dynamic computation graph, ease of debugging, and robust ecosystem of pre-trained models via torchvision. The EfficientNet-B0 architecture was loaded with pre-trained ImageNet weights from the torchvision model zoo.

4.2 Object Detection

YOLOv8 (You Only Look Once, version 8), developed by Ultralytics, was used for the object detection phase. It supports real-time multi-class detection with anchor-free architecture and is available as an open-source Python package.

4.3 Backend API

FastAPI was chosen for the backend REST API due to its support for asynchronous ASGI-based request handling and significantly lower latency compared to the synchronous Flask framework. Internal benchmarking revealed approximately three times lower mean response latency under concurrent load. The API accepts multipart-encoded image uploads, performs preprocessing and inference, and returns predictions enriched with disease descriptions and treatment recommendations.

4.4 Frontend

The web application interface was built using HTML5, CSS3, and vanilla JavaScript. A card-style upload panel allows users to select an image file and submit it for analysis. JavaScript handles the image submission to the FastAPI backend asynchronously, and results are rendered inline without a page reload, displaying the disease name, confidence score (as an animated progress bar), description, and recommended treatment steps.

4.5 Dataset

The PlantVillage dataset was the sole dataset used for training and evaluation. It is publicly available and has become the standard benchmark in computational plant pathology, containing over 54,000 leaf images across 38 disease and healthy categories.

5. RESULTS AND FINDINGS

5.1 Training Dynamics

Training loss started at approximately 2,340 in the first epoch, dropped to around 1,050 by the second epoch, and reached approximately 475 by the end of epoch five — a consistent and stable decline with no signs of divergence. Validation accuracy climbed from approximately 83.5% at epoch one to 93.1% at epoch five, with the steepest improvement occurring between the first and second epochs. This pattern is characteristic of transfer learning: the model rapidly adapts pre-learned ImageNet features (edges, textures, colour gradients) to the domain of plant disease images, followed by slower fine-tuning of disease-

specific indicators. Crucially, validation loss showed no upward trend during the five epochs, indicating that overfitting was not a concern within this training budget.

The validation accuracy had not plateaued by epoch five, indicating that further training would yield additional gains. This is consistent with Atila et al. [7], who reported that EfficientNet-B0 models trained for 20+ epochs on PlantVillage can exceed 98% validation accuracy. The five-epoch results presented here therefore represent a strong baseline, not the model's performance ceiling.

5.2 Quantitative Performance

Table I summarizes the training configuration and final performance metrics at the end of epoch five.

Metric	Value
Training Epochs	5 (computational constraint)
Validation Accuracy	93.1%
Final Training Loss	~475
Batch Size	32
Learning Rate	1×10^{-4}
Optimiser	Adam

Table I. Training Configuration and Final Metrics

5.3 Architecture Comparison at Five Epochs

To provide a fair benchmark, all four architectures — VGG16, ResNet-50, MobileNetV2, and EfficientNet-B0 — were trained under identical conditions: the same dataset split, the same five-epoch budget, the same Adam optimizer settings, and the same batch size. Results are presented in Table II.

Model	Parameters	Validation Accuracy
VGG16	138.4 M	92.0%
ResNet-50	25.6 M	93.0%
MobileNetV2	3.4 M	93.0%

EfficientNet-B0	5.3 M	93.1%

Table II. Architecture Comparison at Five Epochs

EfficientNet-B0 achieved 93.1% validation accuracy using only 5.3 million parameters. VGG16 reached just 92.0% despite its 138.4 million parameters, reflecting the inherent difficulty of converging such a large model within five epochs. ResNet-50 and MobileNetV2 both achieved 93.0%, marginally behind EfficientNet-B0, but with higher or comparable parameter counts. EfficientNet-B0 thus delivers the best accuracy-to-efficiency ratio among all architectures tested, confirming its suitability for resource-constrained deployment scenarios.

6. SYSTEM DESIGN AND DEPLOYMENT

6.1 Backend Architecture — FastAPI

The trained EfficientNet-B0 model was integrated into a production-ready REST API built with FastAPI, a modern Python web framework supporting asynchronous ASGI-based request handling. FastAPI was selected over the synchronous Flask framework following internal benchmarking, which recorded approximately three times lower mean response latency under concurrent load (Table III). The API endpoint accepts a multipart-encoded image upload, applies the identical preprocessing transformations used during model training, executes forward inference, and returns the predicted class label along with its softmax confidence score. Each prediction is immediately enriched via a knowledge-base lookup that maps the predicted class to a plain-language disease description and a set of evidence-based treatment recommendations, transforming the system from a bare classifier into a practical diagnostic assistant.

Concurrent Load	Flask (Sync)	FastAPI
10 req/sec	45 ms	15 ms
50 req/sec	120 ms	28 ms
100 req/sec	350 ms	42 ms

Table III. Mean Response Latency: Flask vs. FastAPI

6.2 Web Application Interface

The user interface was built using HTML5, CSS3, and plain JavaScript. The main element is a card-style upload panel. Upon selecting a file and submitting, JavaScript asynchronously sends the image to the FastAPI backend without triggering a page reload. Results appear inline: the disease name, a confidence score displayed as an animated progress bar, a brief description, and recommended treatment steps. The system demonstrated accurate real-world performance — for instance, correctly identifying Apple Scab in a field photograph with 92.53% confidence and recommending the use of fungicides and removal of infected leaves.

7. DISCUSSION

7.1 Architectural Alignment with Task Structure

A key insight from this project is the importance of selecting a model architecture that genuinely fits the problem structure. YOLOv8 was designed for scenes requiring simultaneous object localization and classification. Applied to PlantVillage images — individual leaves requiring only a class label — the detection setup introduced unnecessary complexity (bounding-box labels, anchor mechanisms, multi-scale feature extraction) without contributing to diagnostic accuracy. Switching to EfficientNet-B0 resolved this mismatch and produced substantially better results, reinforcing the principle that simplicity and task-alignment often outperform architectural sophistication.

7.2 Convergence Behaviour and Transfer Learning

The learning curves observed in this study exhibit the hallmark pattern of effective transfer learning: a sharp initial drop in training loss as the model rapidly adapts general ImageNet features to plant disease imagery, followed by slower, steady improvement as disease-specific indicators (lesion patterns, leaf deformities, colour anomalies) are fine-tuned. The absence of any upward trend in validation loss across the five epochs confirms robust generalization with no overfitting — a particularly notable outcome given the constrained training budget.

7.3 Limitations

Three key limitations must be acknowledged. First, training was limited to five epochs due to compute constraints; published work using the same architecture trained for 20+ epochs reports validation accuracy exceeding 98%, so the 93.1% reported here is a conservative baseline. Second, the model expects a single, isolated leaf per image. Multi-leaf photographs, complex backgrounds, or non-plant objects will degrade prediction quality, and the softmax output layer will always assign one of the 38 classes even when the input is invalid. Third, the PlantVillage dataset was collected under controlled laboratory conditions, which may not fully represent real-world field imagery — model performance on genuinely field-captured photographs may be lower than the reported validation accuracy suggests.

8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project successfully demonstrates the integration of a high-performance deep learning classifier with an accessible web application for automated plant disease diagnosis. Despite the constraint of only five training epochs and limited computing resources, EfficientNet-B0 achieved 93.1% validation accuracy on the PlantVillage benchmark, with rapid convergence and strong generalization through transfer learning. A comparative evaluation against VGG16, ResNet-50, and MobileNetV2 confirmed that EfficientNet-B0 delivers competitive accuracy with significantly fewer parameters, making it the most suitable choice for real-world deployment. The FastAPI web application enables farmers and agronomists without technical backgrounds to obtain real-time disease diagnoses and treatment recommendations from a simple leaf photograph.

8.2 Future Scope

Several directions are planned for future development:

1. Extended training using advanced learning rate schedules such as cosine annealing, targeting validation accuracy above 98%.
2. Model conversion to ONNX or TensorFlow Lite for offline deployment on Android devices, enabling use in areas with limited internet connectivity.

3. Addition of an out-of-distribution detection module to identify non-leaf inputs and improve robustness under messy field conditions.
4. Dataset expansion to include region-specific crops and a greater proportion of field-captured images for improved global coverage.
5. Integration of a lightweight YOLO front-end to first localize the leaf region within an image before passing it to EfficientNet-B0, improving performance on cluttered scenes.
6. Implementation of Grad-CAM visualization to generate heatmaps indicating which regions of a leaf influenced the model's prediction, building interpretability and farmer trust.

REFERENCES

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," Frontiers in Plant Science, vol. 7, p. 1419, 2016.

[2] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. Int. Conf. Machine Learning (ICML), 2019, pp. 6105–6114.

[3] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," arXiv preprint arXiv:1409.1556, 2014.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.

[5] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," arXiv preprint arXiv:1804.02767, 2018.

[6] D. Singh et al., "PlantDoc: A dataset for visual plant disease detection," in Proc. 7th ACM IKDD CoDS and 25th COMAD, 2020.

[7] U. Atila, M. Uçar, K. Akyol, and E. Uçar, "Plant leaf disease classification using EfficientNet deep learning model," Ecological Informatics, vol. 61, p. 101182, 2021.

[8] A. Ramcharan, K. Baranowski, P. McCloskey, B. Ahmed, J. Legg, and D. Hughes, "Deep learning for image-based cassava disease detection," Frontiers in Plant Science, vol. 8, p. 1852, 2017.

[9] M. Arsenovic, M. Karanovic, S. Sladojevic, A. Anderla, and D. Stefanovic, "Solving current limitations of deep learning based approaches for plant disease detection," Symmetry, vol. 11, no. 7, p. 939, 2019.

[10] P. W. Chin, K. W. Ng, and N. Palanichamy, "Plant disease detection and classification using deep learning methods: A comparison study," Journal of Informatics and Web Engineering, vol. 3, no. 1, pp. 1–18, 2024.

APPENDICES

Appendix A: Model Architecture Summary

EfficientNet-B0 is the baseline model in the EfficientNet family. It takes input of size 224×224×3 and passes it through a series of Mobile Inverted Bottleneck (MBConv) blocks with squeeze-and-excitation optimization. The final feature vector of dimension 1,280 is passed to a fully connected classification head. In this project, the original 1,000-class head was replaced with a 38-class output layer for the PlantVillage classification task.

Appendix B: Training Environment

All training and evaluation was performed on a standard consumer-grade machine with limited GPU access. Python 3.10 was used with PyTorch 2.x, torchvision, and the Ultralytics YOLOv8 package. The FastAPI application was served using Uvicorn and tested locally using standard browser-based HTTP requests.

Appendix C: Knowledge Base Structure

The knowledge base is a Python dictionary keyed by disease class name (e.g., 'Apple___Apple_scab'). Each entry contains a plain-language disease description and a list of treatment recommendations. Upon inference, the predicted class label is used to retrieve the corresponding entry, which is appended to the API response alongside the confidence score.